Sara Batatawala || 221A004 || 4

1. Import dependencies

```
import os
import gymnasium as gym
import numpy as np
```

2. Load environment

```
env = gym.make("Taxi-v3",render_mode="rgb_array")
state = env.reset()
state
```

```
(392, {'prob': 1.0, 'action_mask': array([1, 1, 0, 1, 0, 0], dtype=int8)})
```

```
env.observation_space.n
```

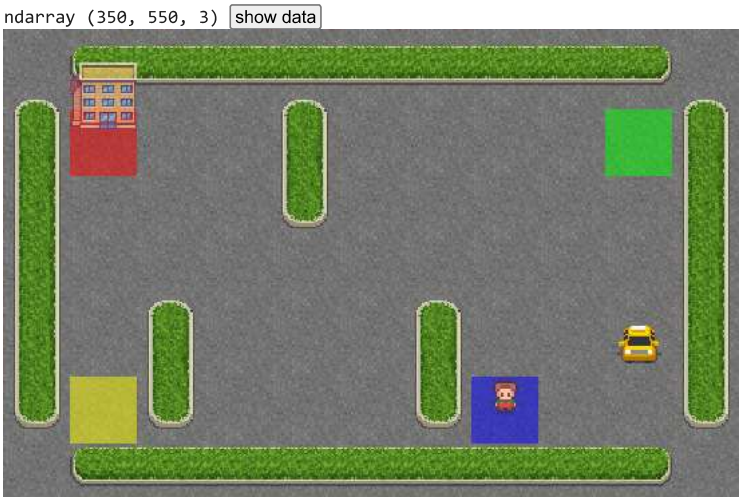
```
np.int64(500)
```

```
env.action_space.n
```

```
np.int64(6)
```

```
env.render()
```

ndarray (350, 550, 3) show data



```
env.step(4)
```

```
(392,
 -10,
 False,
 False,
 {'prob': 1.0, 'action_mask': array([1, 1, 0, 1, 0, 0], dtype=int8)})
```

Possible actions

down(0), up(1), right(2), left (3), pick-up(4) and drop-off(5)

```
n_states=env.observation_space.n
n_actions=env.action_space.n

n_actions
```

```
np.int64(6)
```

How good does behaving completely random do ?

```
state=env.reset()
counter=0
g=0
reward=None
```

```
while reward != 20:
    state,reward,done,info=env.step(env.action_space.sample())
    counter+=1
    g+=reward
```

```
print("Solved in {} Steps with a total reward of {}".format(counter,g))
```

```
Solved in 495 Steps with a total reward of -1977
```

Let's look at just one episode and see how the Q values change after each step using the formula below

$$Q_{n+1} = Q_n + \alpha_n \left(R_n - Q_n \right)$$

```
Q=np.zeros([n_states,n_actions])
print(n_actions)
```

```
6
```

```
episodes=1
G=0
alpha=0.618
```

```
for episode in range(1, episodes+1):
  steps_q_learning = 0
  G, reward=0,0
  state, _ = env.reset()
  state = int(state)
  finalState=state
  print(f"Initial state {state}")
  while reward != 20:
      action=np.argmax(Q[state])
      state2_tuple, reward, terminated, truncated, info  = env.step(action)
      state2 = int(state2_tuple)
      Q[state, action] = Q[state, action] + alpha * (reward + np.max(Q[state2]) - Q[state, action])
      G += reward
      steps_q_learning += 1
      state=state2
  print(f"Episode {episode} solved in {steps_q_learning} steps with a total reward of {G}")
```

```
Initial state 332
Episode 1 solved in 14 steps with a total reward of 7
```

```
finalState
```

```
332
```

```
G
```

```
7
```

After 1 episode of implementing Q learnign total reward is better than random approach

This way we get the optimal policy solution (agent learnt the optimal policy)